

Project Title: Phishing-Resistant -Authentication with FIDO2 Passkeys in a Self Hosted Identity Provider

Author : NOOR

Supervised by: Dr. Mohammed Ammar

01

Executive Summary & Core Objective

Modern public web services face critical vulnerabilities stemming from credential .stuffing, brute-force attacks, and adversary-in-the-middle (AitM) phishing proxies Traditional authentication frameworks rely on shared secrets (such as static passwords or symmetric OTP tokens) that are vulnerable to interception



The W3C Web Authentication (WebAuthn Level 2) specification operating collaboratively with FIDO2 and CTAP2 protocols, establishes an asymmetric public-key infrastructure natively integrated into web clients. By replacing shared secrets with hardware-isolated private keys bound strictly to .origin scopes, WebAuthn provides mathematically guaranteed phishing resistance This report provides an exhaustive technical analysis of four foundational sections specified under W3C WebAuthn Level 2: Section 7.1, Section 7.2, Section 5.8.1, and Section 6.1

Section 7.1: Registering a New Credential (Registration Ceremony)

Section 7.1 governs the programmatic flow through which a user provisions a new public-key credential bound to a Relying Party (RP)

Technical Execution Flow

echnical Execution Flow:

Cryptographic Challenge Generation: The RP (Identity Provider Server) produces an unpredictable, cryptographically secure binary challenge (challenge) of at least 16 bytes. This value is temporarily cached on the server to enforce ceremony freshness and prevent replay attacks.

Client Invocation: The web application invokes the browser API via navigator.credentials.create(), supplying an options dictionary (PublicKeyCredentialCreationOptions) containing:
rp: Identity parameters of the server (id and name).

user: Unique identifier (id), display name, and handle.

pubKeyCredParams: Acceptable cryptographic algorithms (e.g., ES256, Ed25519, RS256).
authenticatorSelection: Hardware and verification parameters.

Authenticator Keypair Generation:

The platform or external authenticator prompts the user for local verification (biometrics or device PIN). Once approved, the authenticator generates an asymmetric keypair strictly scoped to the RP ID:

- **Private Key**: Stored within hardware-isolated boundaries (TPM, Secure Enclave) and never leaves the device.
- **Public Key:** Encoded into standard COSE (CBOR Object Signing and Encryption) structure.

Server-Side Validation:

The RP validates the incoming payload by:

- Verifying the returned challenge matches the cached challenge.
- Verifying that clientDataJSON.origin strictly matches the expected host scheme and fully qualified domain name.
- Extracting the public key and storing it in the user database alongside the generated credential ID.

Section 7.2: Verifying an Authentication Assertion (Assertion Ceremony)

Section 7.2 formalizes the zero-password authentication ceremony, proving possession of the private key without exposing credentials or biometric data.

Technical Execution Flow:
1. Assertion Request Initiation:
Upon login attempt, the RP generates a fresh challenge and issues PublicKeyCredentialRequestOptions, specifying allowed credential IDs and verification requirements.

2. Client Invocation & Verification:
The client executes navigator.credentials.get(), routing the context to the authenticator. The user verifies locally via biometrics or PIN on-device; raw biometrics are never exposed.

3. Cryptographic Assertion Generation:
The authenticator signs the challenge and client data using the isolated private key:
Signature = Sign(PrivateKey, authData + Hash(clientDataJSON))

4. Server-Side Verification Algorithm:
The server receives the assertion signature, authData, and client data, and validates the following:
• Challenge: Matches the active session.
• Origin: RP ID hash matches the expected domain.
• Flags: Verifies user presence (UP) and user verification (UV).
• Cryptographic Signature: Decrypts successfully with the stored public key.
• Sign Count: Increments monotonically to detect credential cloning.

Section 5.8.1: Authenticator Selection Criteria

Configured via AuthenticatorSelectionCriteria to enforce hardware policies:
• authenticatorAttachment: Restricts authenticators to built-in platform sensors ("platform", e.g., Windows Hello, Touch ID) or external hardware tokens ("cross-platform", e.g., USB YubiKeys).
• residentKey / requireResidentKey: Enforces storage of Discoverable Credentials (Passkeys), enabling one-touch, username-less login flows.
• userVerification: Dictates whether biometric/PIN evaluation is strictly enforced ("required"), preferred ("preferred"), or bypassed ("discouraged").

Section 6.1: Authenticator Data (authData) Structure

The authData is a compact binary payload returned by the authenticator:
• rpIdHash (32 Bytes): SHA-256 hash of the effective domain. The browser enforces this check, ensuring credentials cannot be sent to phishing or spoofed origins.
• flags (1 Byte): Bitmask signaling ceremony states (UP: User Present, UV: User Verified, AT: Attested Credential Data present).
• signCount (4 Bytes): A monotonic 32-bit counter incremented with each use to detect cloned authenticators.
• Attested Credential Data: Contains the credential ID and public key in COSE format (present during registration).

Architectural Conclusion & Application in Self-Hosted IdP

Implementing WebAuthn Level 2 inside an enterprise Self-Hosted Identity Provider (such as Keycloak or Authentik) provides comprehensive defense-in-depth

Zero Attack Surface for Credential Theft: Eliminates server-side storage of sensitive passwords; only mathematically public keys reside in the database

Cryptographic Anti-Phishing: Enforced browser-level origin validation prevents unauthorized harvesting across arbitrary network topologies

Regulatory & Modern Identity Compliance: Delivers modern authentication standards aligned with FIDO Alliance, NIST SP 800-63B guidelines, and European zero-trust security postures





Thanks